What I am presenting on 9 October, explained from the ground up

Siddharth Mishra (notes for myself)

9 October 2026

These notes are for me. They explain every idea behind what I will show the professor, starting from the definitions, so that I can answer questions without looking anything up. The grey boxes say each step again in plain words. Everything here is backed by a file in `~/work/gt/lecc-small/`.

Contents

1 The whole story on one page **1**

2 Background **2**
2.1 Edge colouring and list edge colouring 2
2.2 Line graphs 2
2.3 Tool 1: the kernel method (Galvin) 2
2.4 Tool 2: Alon–Tarsi (the polynomial method) 3

3 Schauz's zero-sum graphs **3**
3.1 Why Alon–Tarsi has only one chance on regular graphs 3
3.2 What I computed 3

4 The theorem: the kernel method needs bipartiteness **4**
4.1 The proof, step by step 4
4.2 What the theorem does and does not say 5

5 The computer proofs **5**
5.1 The idea 5
5.2 The lemmas, each with a one-line reason 5
5.3 The loop 6
5.4 Why I trust the answer 6
5.5 Results 6

6 What is new and what is not **7**

7 How I will present it **7**

8 What we do next **8**

1 The whole story on one page

1. **The problem.** The List Edge Colouring Conjecture (LECC) says that if the edges of a graph can be properly coloured with k colours, then they can also be coloured when every edge must pick its colour from its own private list of k colours.

2. **The open cases I attacked.** Schauz (2017) tried to check the LECC by computer for all graphs on at most 10 vertices. His method breaks down on certain regular graphs, which he called *zero-sum* graphs. I reproduced his count exactly and removed the graphs that known theorems already handle. That leaves **52 open graphs**: 3 on 8 vertices and 49 on 10 vertices.

3. **A theorem.** The other classical tool, Galvin's kernel method, also fails on all of them. I proved why: *for a regular graph, the kernel method works if and only if the graph is bipartite.* None of the 52 is bipartite.

4. **Computer proofs.** Since both classical tools fail, I built a third method: a computer search, pruned by small lemmas, which shows that no bad list assignment exists. It proves the LECC for **6 of the 52 graphs** (all 5 of degree 3 and one of degree 4). Each proof comes with a certificate that independent code re-checks.

5. **Novelty.** The theorem and the 6 settled graphs are not in any literature I could find. The census is a reproduction, the odd-ring lemma is known, and the search method is in the spirit of published SAT-based work.

6. **Next.** Settle the remaining 46 graphs (the obstacle is memory, and I have a plan), finish the literature check, add formal proof certificates, and ask the professor which direction he prefers.

2 Background

2.1 Edge colouring and list edge colouring

A *proper edge colouring* gives every edge a colour so that two edges meeting at a vertex get different colours. The smallest number of colours that works is the *chromatic index* $\chi'(G)$. Every vertex of degree Δ needs Δ different colours on its edges, so $\chi'(G) \geq \Delta(G)$. Vizing's theorem says $\chi'(G)$ is always Δ or $\Delta + 1$. Graphs with $\chi' = \Delta$ are called *class 1*.

In *list* edge colouring, every edge e comes with its own list $L(e)$ of allowed colours, and the colouring must take each edge's colour from its list. The *list chromatic index* $\mathrm{ch}'(G)$ is the smallest k such that every assignment of lists of size k can be coloured from. Giving every edge the same list $\{1, \ldots, k\}$ is ordinary colouring, so $\mathrm{ch}'(G) \geq \chi'(G)$ always.

Definition 1 (LECC)**.** For every graph G, $\mathrm{ch}'(G) = \chi'(G)$.

In plain words. Different lists never make edge colouring harder than identical lists. This is open in general. It is known for bipartite graphs (Galvin 1995), for complete graphs of certain orders, for planar graphs of large degree, and a few other classes.

2.2 Line graphs

The *line graph* $L(G)$ has one vertex for each edge of G, and two of its vertices are adjacent when the edges share an endpoint. A proper edge colouring of G is exactly a proper vertex colouring of $L(G)$. So the LECC asks whether $L(G)$ is $\chi'(G)$-choosable as a vertex list colouring problem. The computer works with $L(G)$.

If G is k-regular (every vertex has degree k) with $|E|$ edges, then each edge meets $2(k-1)$ others, so $L(G)$ has $|E|$ vertices and $|E|(k-1)$ edges. This count matters twice below.

2.3 Tool 1: the kernel method (Galvin)

Orient each edge of a graph H, giving a directed graph D. A *kernel* is a set K of vertices with no arcs inside it, such that every vertex outside K has an arc into K. D is *kernel-perfect* if every induced subgraph has a kernel.

Lemma 2 (Bondy, Boppana, Siegel; Lemma 1 of the Dinitz chapter in *Proofs from THE BOOK*)**.** *If D is a kernel-perfect orientation of H and every vertex v has more colours in its list than its out-degree, $|L(v)| > d^+(v)$, then H can be coloured from the lists.*

In plain words. Take the vertices that have colour 1 in their list. Their kernel gets colour 1. Everyone else who wanted colour 1 loses it, but each of them also loses an out-arc (it points into the kernel), so the inequality survives. Repeat with the next colour.

Galvin's proof of the Dinitz conjecture finds, for every bipartite G, a kernel-perfect orientation of $L(G)$ with every out-degree at most $\Delta - 1$. With lists of size Δ the lemma then colours $L(G)$. That proves the LECC for bipartite graphs.

2.4 Tool 2: Alon–Tarsi (the polynomial method)

The *graph polynomial* of H is $P_H = \prod_{uv \in E(H)} (x_u - x_v)$. If some monomial $\prod_v x_v^{t_v}$ has a nonzero coefficient and every $t_v < |L(v)|$, the Combinatorial Nullstellensatz colours H from the lists. Alon and Tarsi showed the coefficient equals, up to sign, $EE - EO$ for any orientation with out-degrees t, where EE and EO count the even and odd Eulerian subgraphs of that orientation (arc sets where every vertex has in-degree equal to out-degree).

In plain words. A monomial is a recipe "vertex v may use up to t_v out-arcs". If its coefficient is not zero, any lists with more than t_v colours at each v can be coloured from. Counting Eulerian subgraphs is a second, independent way to compute that coefficient, which I use for checking.

3 Schauz's zero-sum graphs

3.1 Why Alon–Tarsi has only one chance on regular graphs

Let G be k-regular and class 1, and give every edge a list of size k. Alon–Tarsi needs a monomial of $P_{L(G)}$ with every exponent at most $k - 1$. The monomial's degree equals the number of edges of $L(G)$, which is $|E|(k-1)$, and there are $|E|$ variables. The only way to reach total $|E|(k-1)$ with every exponent at most $k-1$ is *every exponent exactly* $k-1$. So there is exactly one monomial to test, $\prod_e x_e^{k-1}$.

Its coefficient is a nonzero constant times

$$S(G) = \sum_{\text{proper } k\text{-edge-colourings } c} \prod_v \operatorname{sgn}(\pi_v(c)),$$

where $\pi_v(c)$ is the permutation at v that sends the edges at v (in a fixed order) to their colours. G is a **zero-sum graph** if $S(G) = 0$. Then Alon–Tarsi says nothing.

3.2 What I computed

- **Census (R1).** I enumerated every connected regular graph on 8 and 10 vertices with nauty and computed $S(G)$. I found 3 zero-sum graphs on 8 vertices and 51 on 10 vertices (by degree 3, 4, 5, 6, 7: 5, 17, 18, 8, 3). This matches Schauz's paper exactly.
- **Filter (R2).** I removed graphs that known theorems already cover (bipartite, planar and 2-connected, bipartite after deleting a vertex, no odd cycle of length 5 or more, complete). Only 2 of the 51 were covered (both bipartite). So **52 graphs are open**: on 8 vertices one each of degree 3, 4 and 5; on 10 vertices 4 of degree 3, 17 of degree 4, 17 of degree 5, 8 of degree 6 and 3 of degree 7.

4 The theorem: the kernel method needs bipartiteness

Theorem 3 (R5)**.** *Let G be a simple k-regular graph. Then $L(G)$ has a kernel-perfect orientation with every out-degree at most $k-1$ if and only if G is bipartite.*

"If" is Galvin. "Only if" is mine. So for any non-bipartite regular graph, the kernel method with Δ colours cannot work, whatever the true answer is. That covers all 52.

4.1 The proof, step by step

Assume D is a kernel-perfect orientation of $L(G)$ with $d^+(e) \le k-1$ for every edge e.

Step 1: ranks. The k edges at a vertex u form a clique in $L(G)$. A clique in a kernel-perfect digraph must be a transitive tournament (a clean ranking), otherwise some sub-clique has no kernel. So at each u the edges are ranked. Let $r_u(v) \in \{0, \dots, k-1\}$ be the rank of edge uv at u, with 0 the best, and let every arc point to the better edge. Edge uv points to the edges ranked above it at u and at v, so $d^+(uv) = r_u(v) + r_v(u)$.

In plain words. Every vertex lines up its edges from favourite to least favourite. An edge's out-degree counts how many edges beat it at its two ends.

Step 2: no slack. Add the out-degrees over all edges. At each vertex the ranks are $0, 1, \dots, k-1$, which sum to $k(k-1)/2$. So the total is $|V|k(k-1)/2 = |E|(k-1)$. There are $|E|$ edges, each with out-degree at most $k-1$, so *every* out-degree is exactly $k-1$:

$$r_u(v) + r_v(u) = k-1 \quad \text{for every edge } uv.$$

In plain words. The budget is completely used up. If u ranks v highly, then v must rank u lowly, and the other way round.

Step 3: no odd ring. An *odd ring* is an odd cycle $a_1 a_2 \dots a_m$ where every a_i prefers its next neighbour a_{i+1} to its previous one a_{i-1}. If there were one, its edges would form a directed odd cycle in D, and a directed odd cycle has no kernel. So there is no odd ring.

Step 4: an orientation of G itself. Say u "points to" v if u ranks v in its better half (rank below $(k-1)/2$). By Step 2, if u ranks v in its better half, then v ranks u in its worse half, so this is a consistent orientation O of G. If k is odd, the edges of exactly middle rank are left out. Each vertex has exactly one such edge, so they form a perfect matching M_0. Every vertex has the same number of out-edges and in-edges in O (half its non-middle edges each way), so O is *balanced*.

Step 5: following O around a cycle makes a ring. Walk around a cycle using only forward arcs of O and edges of M_0 (never two M_0 edges in a row, since M_0 is a matching). A short calculation with the ranks shows that at every vertex of such a walk, the next neighbour is preferred to the previous one. So such a cycle, if odd, would be an odd ring. By Step 3 there is none.

In plain words. The orientation O is a map of preferences. Any odd cycle that obeys the map is exactly the forbidden odd ring.

Step 6: strong connectivity. In a balanced digraph every arc lies on a directed cycle. So every connected piece of $G - M_0$ is strongly connected under O: you can get from any vertex to any other by following arcs.

Step 7: each piece is bipartite. Suppose a piece is not bipartite, and suppose for now that it has no directed closed walk of odd length. For an arc $x \to y$, pick a directed path from y back to x (it exists by Step 6). Together with the arc it closes a walk, so the path has odd length. Now take an odd cycle of the piece and walk around it. Keep the arcs that point forwards, and replace each arc that points backwards by its odd return path. Each replacement swaps 1 edge for an odd number of edges, so the parity does not change: the result is a directed closed walk of odd length. That contradicts the assumption. So the piece has an odd directed closed walk, and hence an odd directed cycle. Step 5 forbids that. So every piece is bipartite.

Step 8: k even. Then M_0 is empty, so G is just the union of its pieces, and it is bipartite.

Step 9: k odd. Now the pieces are glued together by the matching M_0. Colour each piece with two colours. Build a small graph Γ whose nodes are the pieces and whose edges are the M_0 edges, and label an M_0 edge 1 if its ends got the same colour. G is bipartite exactly when every cycle in Γ has an even label sum. If some cycle had an odd sum, I can follow it through the pieces using directed paths of O (Step 6) and M_0 edges. That gives a real cycle of G that obeys the map of Step 5, and a parity count shows its length is odd. Step 5 forbids that. So G is bipartite. □

In plain words. The tight budget turns preferences into a balanced orientation. A balanced orientation of a connected graph lets you travel everywhere. If the graph had any odd cycle, you could travel around an odd loop that obeys the preferences, and that loop is exactly what kills the kernel.

4.2 What the theorem does and does not say

- It needs *regular* (that is what makes the budget tight) and *simple* (so two edges share at most one vertex). It needs lists of size exactly Δ. With $\Delta + 1$ colours the Petersen graph does have a kernel-perfect orientation (McDonald 2015), so the barrier really is about the tight budget.
- It does not say these graphs fail the LECC. It only says this one tool cannot prove it.
- The computer confirms it on 1923 regular graphs: a suitable orientation exists for exactly the 29 bipartite ones.

5 The computer proofs

5.1 The idea

To show $L(G)$ is k-choosable I must show there is no *bad* list assignment, meaning lists of size k from which no proper colouring exists. There are far too many assignments to try one by one. So a SAT solver searches for a bad assignment while a list of lemmas rules out huge families at once. If the solver proves no candidate is left, the graph is choosable.

5.2 The lemmas, each with a one-line reason

(P) Precheck
For every vertex v, $L(G) - v$ passes Alon–Tarsi with lists of size k. (The computer checks this; deleting a vertex frees up the budget.) So every proper piece of $L(G)$ is k-choosable, and a bad assignment must be "critical".

(C) Criticality
In a bad assignment, every colour in v's list also appears in some neighbour's list. Otherwise colour everything else first (possible by (P)) and give v that colour.

(B) Palette
By (C) each colour appears in at least two lists, so at most $nk/2$ colours are used in total (n vertices, k slots each). The solver only needs that many.

(S) Symmetry
Colour names do not matter, so I rename colours in order of first appearance. This removes an enormous number of duplicate cases.

(R) Reducible colour classes
Look at the set A of vertices whose list contains colour c. If I can give colour c to an independent set I inside A and the rest still passes Alon–Tarsi with the reduced lists, the assignment is colourable. Whether this works depends only on the set A, so each such fact rules out every assignment with a colour class of that shape. I call each such fact a *witness*.

5.3 The loop

The solver proposes a list assignment that obeys all the lemmas. A second solver tries to colour from it. If it succeeds, the colouring is recorded and every assignment it also colours is excluded. Then the loop repeats. When the first solver says "no assignment left" (UNSAT), the graph is choosable. This is a counterexample-guided loop.

5.4 Why I trust the answer

1. **Validation on known cases.** On 79 small random graphs the search agrees with pure brute force over every list assignment. It also finds the known bad cases, for example that L(Petersen) is not 3-choosable.

2. **Certificates.** When the solver says UNSAT, it also reports which witnesses it actually used (its *unsat core*). I save those witnesses, their Alon–Tarsi monomials and all the recorded colourings in a file.

3. **Independent checker.** A separate program, sharing no code with the search, re-checks every Alon–Tarsi fact by counting Eulerian subgraphs (the other side of the Alon–Tarsi theorem), checks every recorded colouring is proper, rebuilds the whole formula itself, and confirms UNSAT with a different SAT solver (Glucose instead of CaDiCaL).

5.5 Results

graph (graph6)	vertices	degree	witnesses used / generated	check
`GCY^B_` (called X)	8	3	892 / 6972	passed
`I?`amQoM?`	10	3	4636 / 73657	passed
`I?`bfAWF?`	10	3	4473 / 75994	passed
`ICOedO[X?`	10	3	5809 / 73302	passed
`I?bEHow[?`	10	3	7723 / 70276	passed
`GEnbvG`	8	4	2235 / 201177	passed

For `GEnbvG` I first tried adding all 201177 witnesses up front. That is 6.4 million clauses, and the solver never finished. The fix was to add witnesses *lazily*: only when a proposed assignment actually has a colour class matching one. Then the search took 30 seconds.

Where it stops today. The next graphs (degree 5 on 8 vertices, and the graphs on 10 vertices of degree 4 and up) run out of memory before the search even starts. The step that computes all Alon–Tarsi coefficients needs more than 7 GB. This is an engineering wall, not a mathematical one.

6 What is new and what is not

Result	Status
Census of zero-sum graphs	Not new. Reproduces Schauz exactly.
Odd-ring lemma	Known (Borodin, Kostochka, Woodall 1998; Tan; Chung).
Theorem R5	Not found in the literature. Checked: Borodin–Kostochka–Woodall 1998, McDonald 2015, McDonald–Puleo 2017, everything citing BKW 1998, Jafari 2026. Not yet checked: Blumenthal's 2016 Auburn thesis and Woodall's 2001 survey. Could be folklore.
LECC for the 6 graphs	New as far as I can find. Schauz lists them as open, no citing paper settles them, no known theorem covers them, and both standard tools provably fail.
The search method	Not new as a method. SAT-based choosability search is published (Kirchweger, Peitl, Seka, Szeider, AAAI 2026, for vertex colouring). I cite it.

One possible worry, already checked. McDonald and Puleo use kernel-perfect orientations on some non-bipartite graphs. That does not contradict R5. They allow pairs of opposite arcs, but on a regular graph the count in Step 2 rules those out. And no non-bipartite regular graph lies in their class of graphs.

7 How I will present it

1. **The target** (1 minute): the LECC, Schauz's attack, zero-sum graphs, 52 open.
2. **The theorem** (3 minutes): statement, the one-line idea, Step 2 and Step 5 if he asks. Say "I have not found it in the literature", not "new". Ask if he knows it.
3. **The computer proofs** (2 minutes): 6 of 52 settled, how they are checked, where the method stops.
4. **The question**: which direction next (below).

Questions he might ask.

- *Why can Alon–Tarsi not use another monomial?* Section 3.1: the edge count forces every exponent to be $k-1$.
- *Why must the ranks at a vertex be a clean ranking?* A clique in a kernel-perfect digraph is a transitive tournament; a directed triangle inside it has no kernel.
- *Is the computer proof trustworthy?* Every lemma has a one-line proof, the search agrees with brute force on small graphs, and an independent program with a different solver re-checks every certificate.
- *What remains to make it airtight?* A DRAT proof trace for the final SAT step, and a second human reading of the lemmas.

8 What we do next

1. **Settle the remaining 46 graphs.** Replace the memory-hungry coefficient step: find each Alon–Tarsi fact by searching for an orientation with the right out-degrees and checking $EE \neq EO$ with the fast Eulerian counter I wrote this week (it uses little memory). Every fact found is still valid, so the proofs stay sound. Order: the degree-5 graph on 8 vertices, then the 10-vertex graphs by increasing degree.

2. **Formal certificates.** Have the SAT solver write a DRAT proof of the final UNSAT and check it with `drat-trim`.

3. **Finish the novelty check for R5.** Read Blumenthal's thesis and Woodall's survey, and ask experts on Galvin's method whether the theorem is known.

4. **Extend R5** (if the professor prefers the theoretical direction): multigraphs, non-regular graphs where the budget is not tight, and lists of size $\Delta + 1$.

5. **Longer term.** If all 52 fall, the remaining gaps in Schauz's programme (non-regular and odd-order graphs on at most 10 vertices) are the natural next target.

6. **Ask the professor** to choose: (a) push the computer method through all 52, (b) develop R5 into a short paper, or (c) switch to the odd-torus problem $\chi_\ell(C_{2m+1} \,\square\, C_{2n+1}) = 3$. Also agree what the 16 October email should contain, since I am away that week.

Where everything lives

- `lecc-small/RESULTS.md`: every verified result with the script that produced it.
- `lecc-small/03-regular-bipartite-theorem.md`: the theorem.
- `lecc-small/04-computer-proofs.md`: the computer method.
- `lecc-small/05-novelty-check.md`: the literature check.
- `lecc-small/data/cores/` and `data/lazy/`: certificates and check outputs.